

# **Applied Cryptography**

## **Week 7/ Final Exam**

Student Name: **Mikheil Davidovi**  
Class: **Applied Cryptography**

Submission Date: **November 25, 2025**

### ***Tools and Materials:***

Command Prompt

OpenSSL

Windows PowerShell

PyCharm

WireShark

GPG

## **Task 1 – Encrypted Messaging App Prototype**

***Goal: Implement a mini encrypted messaging system that supports both RSA and AES.***

### ***Python code:***

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

def generate_rsa_keys():
    key = RSA.generate(2048)
    private_key = key.export_key()
    public_key = key.publickey().export_key()
    with open("userA_private.pem", "wb") as f:
        f.write(private_key)
    with open("userA_public.pem", "wb") as f:
        f.write(public_key)

def encrypt_message():
    message = open("message.txt", "r").read().encode()
    aes_key = get_random_bytes(32)
    cipher_aes = AES.new(aes_key, AES.MODE_EAX)
    ciphertext, tag = cipher_aes.encrypt_and_digest(message)
    with open("encrypted_message.bin", "wb") as f:
```

```

        f.write(cipher_aes.nonce + tag + ciphertext)
    public_key = RSA.import_key(open("userA_public.pem", "rb").read())
    cipher_rsa = PKCS1_OAEP.new(public_key)
    encrypted_key = cipher_rsa.encrypt(aes_key)
    with open("aes_key_encrypted.bin", "wb") as f:
        f.write(encrypted_key)

def decrypt_message():
    private_key = RSA.import_key(open("userA_private.pem", "rb").read())
    cipher_rsa = PKCS1_OAEP.new(private_key)
    encrypted_key = open("aes_key_encrypted.bin", "rb").read()
    aes_key = cipher_rsa.decrypt(encrypted_key)
    data = open("encrypted_message.bin", "rb").read()
    nonce = data[:16]
    tag = data[16:32]
    ciphertext = data[32:]
    cipher_aes = AES.new(aes_key, AES.MODE_EAX, nonce)
    message = cipher_aes.decrypt_and_verify(ciphertext, tag)
    with open("decrypted_message.txt", "wb") as f:
        f.write(message)

if __name__ == "__main__":
    generate_rsa_keys()
    encrypt_message()
    decrypt_message()

```

### ***Encryption Flow Summary:***

#### **Overview:**

This prototype demonstrates a simple encrypted messaging system using a hybrid encryption model:

- AES-256 (symmetric) is used to encrypt the actual message.
- RSA-2048 (asymmetric) is used to encrypt the AES key.

**This combines the efficiency of symmetric encryption with the key-exchange security of asymmetric encryption.**

***Flow:***

### **1. RSA Key Generation**

- User A generates an RSA-2048 key pair.
- Public key - shared with User B
- Private key - kept secret

### **2. AES Key Creation**

- User B generates a random AES-256 key to encrypt the message.

### **3. Message Encryption (AES)**

- User B encrypts the plaintext using AES-256 in EAX mode.
- Output: encrypted\_message.bin

### **4. AES Key Encryption (RSA)**

- User B encrypts the AES key using User A's RSA public key.
- Output: aes\_key\_encrypted.bin

### **5. AES Key Decryption**

- User A decrypts the AES key using their RSA private key.

### **6. Message Decryption (AES)**

- User A decrypts the message using the recovered AES key.
- Output: decrypted\_message.txt

***Artifacts:***

- message.txt
- encrypted\_message.bin
- aes\_key\_encrypted.bin
- decrypted\_message.txt
- Task1.md

## Task 2 – Secure File Exchange Using RSA + AES

**Objective:** Demonstrate hybrid encryption protocol covered in Week 2 & Week 4

**Scenario:** Alice wants to send Bob a secret file securely.

### 1. Generate an RSA key pair for Bob

**Commands:**

```
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048  
openssl pkey -in private.pem -pubout -out public.pem
```

### 2. Alice creates a plaintext message in a file called alice\_message.txt

**Command:**

```
echo "Curiouser and curiouser!" > alice_message.txt
```

### 3. Alice generates a random AES-256 key and IV

**Commands (CMD):**

```
openssl rand -out aes_key.bin 32  
openssl rand -out iv.bin 16
```

**Commands (PowerShell):**

```
(Get-Content "C:\Users\Ronin\aes_key.bin" -Encoding Byte | ForEach-Object {  
$_.ToString("X2") }) -join "" | Set-Content "C:\Users\Ronin\aes_key.hex"  
(Get-Content "C:\Users\Ronin\iv.bin" -Encoding Byte | ForEach-Object { $_.ToString("X2") })  
-join "" | Set-Content "C:\Users\Ronin\iv.hex"
```

### 4. Encrypt the file (alice\_message.txt) using AES-256 with the generated key and IV

**Command:**

```
openssl enc -aes-256-cbc -in "C:\Users\Ronin\alice_message.txt" -out  
"C:\Users\Ronin\encrypted_file.bin" -K (Get-Content "C:\Users\Ronin\aes_key.hex") -iv  
(Get-Content "C:\Users\Ronin\iv.hex")
```

#### **5. Encrypt the AES key using Bob's RSA public key**

##### **Command:**

```
openssl pkeyutl -encrypt -inkey public.pem -pubin -in aes_key.bin -out  
aes_key_encrypted.bin -pkeyopt rsa_padding_mode:oaep
```

#### **6. Bob decrypts the AES key using his RSA private key**

#### **7. Bob uses the decrypted AES key and IV to decrypt the file and recover the original message**

##### **Commands:**

```
openssl pkeyutl -decrypt -inkey private.pem -in aes_key_encrypted.bin -out  
aes_key_decrypted.bin -pkeyopt rsa_padding_mode:oaep
```

```
(Get-Content "C:\Users\Ronin\aes_key_decrypted.bin" -Encoding Byte | ForEach-Object {  
$_.ToString("X2") }) -join "" | Set-Content "C:\Users\Ronin\aes_key_decrypted.hex"
```

```
(Get-Content "C:\Users\Ronin\iv.bin" -Encoding Byte | ForEach-Object { $_.ToString("X2") })  
-join "" | Set-Content "C:\Users\Ronin\iv.hex"
```

```
openssl enc -d -aes-256-cbc -in "C:\Users\Ronin\encrypted_file.bin" -out  
"C:\Users\Ronin\decrypted_message.txt" -K (Get-Content  
"C:\Users\Ronin\aes_key_decrypted.hex") -iv (Get-Content "C:\Users\Ronin\iv.hex")
```

#### **8. After decryption, compute the SHA-256 hash of the file and compare it to the original hash for integrity verification**

##### **Command:**

```
openssl dgst -sha256 alice_message.txt > original_sha256.txt  
openssl dgst -sha256 decrypted_message.txt > decrypted_sha256.txt  
fc original_sha256.txt decrypted_sha256.txt
```

## Output:

```
C:\Users\Ronin>fc original_sha256.txt decrypted_sha256.txt
Comparing files original_sha256.txt and DECRYPTED_SHA256.TXT
***** original_sha256.txt
SHA2-256(alice_message.txt)= a6ee2f098f006205f408fb33c376fa053607a8826cf26017a512a8956727ecbd
***** DECRYPTED_SHA256.TXT
SHA2-256(decrypted_message.txt)= a6ee2f098f006205f408fb33c376fa053607a8826cf26017a512a8956727ecbd
*****
```

## **AES vs RSA Comparison:**

**Type:** AES-256 is symmetric (same key for encryption and decryption); RSA-2048 is asymmetric (public/private key pair).

**Speed:** AES is fast and ideal for large files; RSA is slow and unsuitable for bulk data.

**Security:** AES is powerful if the key remains secret; RSA is strong for secure key exchange or small data.

**Use Case:** AES encrypts the actual message or file; RSA is mainly used to securely transmit AES keys.

**Key Size:** AES uses 256-bit keys; RSA uses 2048-bit keys.

**Encryption/Decryption:** AES uses the same key for both; RSA uses different keys for encryption and decryption.

## **Artifacts:**

- alice\_message.txt
- encrypted\_file.bin
- aes\_key\_encrypted.bin
- decrypted\_message.txt
- public.pem
- private.pem
- original\_sha256.txt
- decrypted\_sha256.txt
- Task2.md

## Task 3 – TLS Communication Inspection & Analysis

**Objective:** Analyze HTTPS using tools learned in Week 4

### 1. Connect to any HTTPS website using openssl s\_client

**Command:**

openssl s\_client -connect [www.google.com:443](https://www.google.com)

**Output:**

Microsoft Windows [Version 10.0.19045.6575]

(c) Microsoft Corporation. All rights reserved.

```
C:\WINDOWS\system32>openssl s_client -connect www.google.com:443
```

```
Connecting to 172.217.169.132
```

```
CONNECTED(000001E8)
```

```
depth=2 C=US, O=Google Trust Services LLC, CN=GTS Root R4
```

```
verify error:num=20:unable to get local issuer certificate
```

```
verify return:1
```

```
depth=1 C=US, O=Google Trust Services, CN=WE2
```

```
verify return:1
```

```
depth=0 CN=www.google.com
```

```
verify return:1
```

```
---
```

```
Certificate chain
```

```
0 s:CN=www.google.com
```

```
  i:C=US, O=Google Trust Services, CN=WE2
```

```
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA256
```

```
  v:NotBefore: Oct 27 08:35:51 2025 GMT; NotAfter: Jan 19 08:35:50 2026 GMT
```

```
1 s:C=US, O=Google Trust Services, CN=WE2
```

```
  i:C=US, O=Google Trust Services LLC, CN=GTS Root R4
```

```
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA384
```

```
  v:NotBefore: Dec 13 09:00:00 2023 GMT; NotAfter: Feb 20 14:00:00 2029 GMT
```

```
2 s:C=US, O=Google Trust Services LLC, CN=GTS Root R4
```



i:C=BE, O=GlobalSign nv-sa, OU=Root CA, CN=GlobalSign Root CA  
a:PKEY: EC, (secp384r1); sigalg: sha256WithRSAEncryption  
v:NotBefore: Nov 15 03:43:21 2023 GMT; NotAfter: Jan 28 00:00:42 2028 GMT

---

Server certificate

-----BEGIN CERTIFICATE-----

MIIDlzCCAz2gAwIBAgIRALHjAKHCxBEtELQnnUUxYh0wCgYIKoZIzj0EAwIwOzEL  
MAkGA1UEBhMCVVMxHjAcBgNVBAoTFUdvb2dsZSBUcnVzdCBTZXJ2aWNlc2EMMAoG  
A1UEAxMDV0UyMB4XDTI1MTAyNzA4MzU1MVVoXDTI2MDExOTA4MzU1MFowGTEXMBU  
G  
A1UEAxMOd3d3Lmdvb2dsZS5jb20wWTATBgqhkhjOPQIBBggqhkhjOPQMBBwNCAAR2  
H4svw5h4htzhxpUS+EHu1BD4240eewWTFWmoebFwv5p0KCeWBgUyaacijMqqjU6F  
/B5zLjcrtdr4k0LE1jrYo4ICQjCCAj4wDgYDVR0PAQH/BAQDAgeAMBMGA1UdJQQM  
MAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVR0OBBYEFMcO8r856LeMu  
fky  
EvGs2KVHsydmMB8GA1UdIwQYMBaAFHW+xHeuifZEN33PsWgfHRrr3DRZMFgGCCsG  
AQUFBwEBBEwwSjAhBggrBgEFBQcwAYYYVaHR0cDovL28ucGtpLmdvb2cvd2UyMCUG  
CCsGAQUFBzACHlloHRwOi8vaS5wa2kuZ29vZy93ZTluY3J0MBkGA1UdEQQSMBCC  
Dnd3dy5nb29nbGUuY29tMBMGA1UdIAQMMAowCAYGZ4EMAQIBMDYGA1UdHwQvMC0  
w  
K6ApoCeGJWh0dHA6Ly9jLnBraS5nb29nL3dlMi94dXp0M1BVOUZfdy5jcmwwggEF  
BgorBgEEAdZ5AgQCBIH2BIHzAPEAdwCWI2S/VViXrfdDh2g3CEJ36fA61fak8zZu  
RqQ/D8qpxgAAAZoIBim+AAAEAwBIMEYCIQChorZ4pP8LI52vbpV+VuRz6HaZVR/Q  
ppFa2G3XBildGgIhAMRi/LeiGMb9sVUr7NACZCrIrDwhBTJj3kqt4Je60grrAHYA  
GYbUxyiqb/66A294Kk0BkarOLXlxD67OXXBBLsVMx9QAAAGaJQYpmwAABAMARzBF  
AiBTBRUqUmuR/KZTk/XhDWP+f0ep/4dQASRx8gB72uVmUgIhAMaSzu9tXq12ZCcH  
N/tIN++unLQmXCqhc+g/qQkM3wPxMAoGCCqGSM49BAMCA0gAMEUCIQc4BQcDiQQr  
SxeyVitvtVcuTGoTqwpv+v7/CD3h/Sg04glgKDj0ubx+aURD8VzHZA1G36dWgUuX  
37rlvvf+PpqfWNw=

-----END CERTIFICATE-----

subject=CN=www.google.com

issuer=C=US, O=Google Trust Services, CN=WE2

```

---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: ecdsa_secp256r1_sha256
Negotiated TLS1.3 group: X25519MLKEM768
---
SSL handshake has read 3894 bytes and written 1625 bytes
Verification error: unable to get local issuer certificate
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 256 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 20 (unable to get local issuer certificate)
---
80250000:error:0A000126:SSL routines::unexpected eof while
reading:ssl\record\rec_layer_s3.c:701:
80250000:error:0A000197:SSL routines:SSL_shutdown:shutdown while in
init:ssl\ssl_lib.c:2834:

C:\WINDOWS\system32>

```

## 2. Briefly describe how TLS provides confidentiality and integrity

### **Explanation:**

**Confidentiality:** TLS uses symmetric encryption (AES) to encrypt data transmitted between client and server, ensuring that eavesdroppers cannot read the content. The symmetric session key is securely exchanged using asymmetric encryption (RSA or ECDH) during the handshake.

**Integrity:** TLS ensures data integrity with message authentication codes (MACs) or authenticated encryption (AEAD), which detect any tampering during transmission. If a message is modified in transit, the recipient can detect it and reject the data.

In short, TLS combines encryption for secrecy and MACs for tamper detection to secure communications.

***Artifacts:***

- tls\_summary.txt
- Client.jpg
- Server.jpg
- HelloDone.jpg
- Openssl 1.jpg
- Openssl 2.jpg

## Task 4 – Email Encryption and Signature Simulation (PGP or S/MIME)

**Objective:** Apply Week 5 concepts on email security

**Scenario:** You are securing an internal company message

### 1. Generate key pairs using GPG or OpenSSL for PGP/S/MIME

**Commands:**

```
gpg --full-generate-key  
gpg --export -a alice > alice_pub.asc  
gpg --export -a bob > bob_pub.asc
```

### 2. Sign and encrypt a text file as "Alice"

**Command:**

```
gpg --output signed_encrypted_message.asc --encrypt --sign --recipient bob --local-user  
alice original_message.txt
```

### 3. Verify and decrypt it as "Bob"

**Command:**

```
gpg --output decrypted_message.txt --decrypt signed_encrypted_message.asc
```

**Artifacts:**

- decrypted\_message.txt
- original\_message.txt
- signed\_message.asc
- alice\_pub.asc
- bob\_pub.asc
- signature\_verification.txt

## Task 5 - Hashing & Integrity Check Utility

**Objective:** Reinforce understanding from Week 3 on hashing and data integrity

**Python code:**

```
import hashlib
import json

def compute_hashes(filename):
    data = open(filename, "rb").read()
    return {
        "sha256": hashlib.sha256(data).hexdigest(),
        "sha1": hashlib.sha1(data).hexdigest(),
        "md5": hashlib.md5(data).hexdigest()
    }

def store_hashes(file, hashes):
    with open("hashes.json", "w") as f:
        json.dump({file: hashes}, f, indent=4)

def verify_integrity(original, modified):
    orig = compute_hashes(original)
    mod = compute_hashes(modified)
    print("Integrity:", "PASS" if orig == mod else "FAIL")

if __name__ == "__main__":
    hashes = compute_hashes("original.txt")
    store_hashes("original.txt", hashes)
    verify_integrity("original.txt", "tampered.txt")
```

**Artifacts:**

- hash\_util.py
- original.txt
- tampered.txt
- hashes.json

- Integrity.jpg